



INNOVATION. AUTOMATION. ANALYTICS

PROJECT ON

EV SALES DATASET



ABOUT ME

Keshamoni Prem Kumar
Bachelors of Technology

Why we want to learn Data Analysis ?

- Learning data analysis helps us understand complex information and make smarter decisions.
- In today's world, data is everywhere—from business to healthcare to social media—and being able to interpret it allows us to identify patterns, trends, and opportunities that aren't obvious at first glance.
- This skill turns raw data into meaningful insights that can guide strategies and actions effectively.

Dataset Overview

- The dataset contains 58100 rows and 14 columns which contains information about ride trips.
- Columns include :
brand,model,range_(km),efficiency_(wh/km),battery_(kwh),fastcharge_speed_(kw),weight_(kg)
- Data types are 5 categorical and 5 numerical columns.

Categorical variables: Brand , Model.

Numerical variables:

range_(km),efficiency_(wh/km),battery_(kwh),fastcharge_speed_(kw),weight_(kg).

EXTRACTING DATA USING WEBSCRAPING

```
headers = {"User-Agent": "Mozilla/5.0"}
```

```
brand = []
model = []
Range = []
efficiency = []
battery = []
fastcharge_speed = []
weight = []
acceleration = []
towweight = []
cargo = []
long_distance_total = []
priceperrange = []
market_segment = []
seats = []
```

```
def extract_number(text):
    "Returns only digits / decimal numbers"
    value = re.findall(r"[\d.]+", text)
    return value[0] if value else np.nan
```

```
for page in range(1, 51):
    print(f"Scraping page {page}...")
    url = f"https://ev-database.org/?page={page}"
    response = requests.get(url, headers=headers)
```

```
if response.status_code == 200:
    soup = bs(response.text, "html.parser")
    element = soup.find_all("div", class_="list-item")
```

```
for item in element:
```

```
    # Brand & Model
    a = item.find("a", class_="title")
    if a:
        model_text = a.text.strip()
        model.append(model_text)
        brand.append(re.findall(r"^\w+", model_text)[0])
    else:
        model.append(np.nan)
        brand.append(np.nan)
```

```
    # Range
    c = item.find("span", class_="erange_real")
    Range.append(extract_number(c.text) if c else np.nan)
```

```
    # Efficiency
    d = item.find("span", class_="efficiency")
    efficiency.append(extract_number(d.text) if d else np.nan)
```

```
    # Battery
    h = item.find("span", class_="battery_p")
    battery.append(extract_number(h.text) if h else np.nan)
```

```
    # Fastcharge speed
    i = item.find("span", class_="fastcharge_speed_print")
    fastcharge_speed.append(extract_number(i.text) if i else np.nan)
```

```
    # Weight
    e = item.find("span", class_="weight_p")
    weight.append(extract_number(e.text) if e else np.nan)
```

```
    # Acceleration
    f = item.find("span", class_="acceleration_p")
```

```
    # Acceleration
```

```
    f = item.find("span", class_="acceleration_p")
    acceleration.append(extract_number(f.text) if f else np.nan)
```

```
    # Tow Weight
```

```
    tow = item.find("span", class_="towweight_p")
    towweight.append(extract_number(tow.text) if tow else np.nan)
```

```
    # Cargo
```

```
    k = item.find("span", class_="cargo")
    cargo.append(extract_number(k.text) if k else np.nan)
```

```
    # Long Distance
```

```
    g = item.find("span", class_="long_distance_total")
    long_distance_total.append(extract_number(g.text) if g else np.nan)
```

```
    # Price per range
```

```
    l = item.find("span", class_="priceperrange_p")
    priceperrange.append(extract_number(l.text) if l else np.nan)
```

```
    # Market Segment
```

```
    o = item.find("span", {"data-tooltip": "Market Segment"})
    market_segment.append(o.text.strip() if o else np.nan)
```

```
    # Seats
```

```
    p = item.find("span", {"data-tooltip": "Number of seats"})
    if p:
        seats.append(extract_number(p.text))
```

```
    else:
        seats.append(np.nan)
```

```
# Create DataFrame
```

```
df = pd.DataFrame({
    "Brand": brand,
    "Model": model,
    "Range (km)": Range,
    "Efficiency (Wh/km)": efficiency,
    "Battery (kWh)": battery,
    "Fastcharge Speed (kW)": fastcharge_speed,
    "Weight (kg)": weight,
    "Acceleration (0-100s)": acceleration,
    "Tow Weight (kg)": towweight,
    "Cargo (L)": cargo,
    "Long Distance (km)": long_distance_total,
    "Price/Range (€ per km)": priceperrange,
    "Market Segment": market_segment,
    "Seats": seats
})
```

```
print(df.head())
df.to_csv("ev_database.csv", index=False)
print("CSV File Exported Successfully!")
```



Reading Dataset

```
df=pd.read_csv("ev_database.csv")
df.head()
```

	Brand	Model	Range (km)	Efficiency (Wh/km)	Battery (kWh)	Fastcharge Speed (kW)	Weight (kg)	Acceleration (0-100s)	Tow Weight (kg)	Cargo (L)	Long Distance (km)	Price/Range (€ per km)	Market Segment	Seats
0	MG	MG MG4 Electric 64 kWhMG MG4 Electric 64 kWh	360	171	61.7	115	1726.0	7.9	500.0	363.0	405	102	C	5
1	BMW	BMW iX3 50 xDrive (MY26)BMW iX3 50 xDrive	610	178	108.7	225	2360.0	4.9	2000.0	578.0	742	113	D	5
2	BMW	BMW iX xDrive40BMW iX xDrive40	360	197	71.0	102	2440.0	6.1	2500.0	500.0	388	225	E	5
3	Tesla	Tesla Model Y Long Range AWD (Juniper)Tesla Mo...	455	165	75.0	124	2072.0	4.8	1600.0	971.0	535	122	D	5
4	Fiat	Fiat 500e Hatchback 42 kWhFiat 500e Hatchback ...	235	159	37.3	67	1365.0	9.0	0.0	185.0	261	143	B	4

Basic Checks

INFO OF DATA

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 58100 entries, 0 to 58099
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	brand	58100 non-null	object
1	model	58100 non-null	object
2	range_(km)	58100 non-null	int64
3	efficiency_(wh/km)	58100 non-null	int64
4	battery_(kwh)	58100 non-null	float64
5	fastcharge_speed_(kw)	58100 non-null	int64
6	weight_(kg)	58100 non-null	float64
7	acceleration_(0_100s)	58100 non-null	float64
8	tow_weight_(kg)	55050 non-null	float64
9	cargo_(l)	56750 non-null	float64
10	long_distance_(km)	58100 non-null	int64
11	price/range_(€_per_km)	58100 non-null	int64
12	market_segment	58100 non-null	object
13	seats	58100 non-null	int64

```
dtypes: float64(5), int64(6), object(3)
```

```
memory usage: 6.2+ MB
```

COLUMNS IN DATA

```
col_names=[col.strip().lower().replace(' ','_').replace('-','_') for col in df.columns]
df.columns=col_names
print(df.columns)
```

```
Index(['brand', 'model', 'range_(km)', 'efficiency_(wh/km)', 'battery_(kwh)',
      'fastcharge_speed_(kw)', 'weight_(kg)', 'acceleration_(0_100s)',
      'tow_weight_(kg)', 'cargo_(l)', 'long_distance_(km)',
      'price/range_(€_per_km)', 'market_segment', 'seats'],
      dtype='object')
```


Preprocessing Techniques on Dataset Individual Columns

Checking Missing values

```
df.isnull().sum()
```

```
brand          0
model          0
range_(km)     0
efficiency_(wh/km) 0
battery_(kwh)  0
fastcharge_speed_(kw) 0
weight_(kg)    0
acceleration_(0_100s) 0
tow_weight_(kg) 53
cargo_(l)      22
long_distance_(km) 0
price/range_(€_per_km) 0
market_segment 0
seats          0
dtype: int64
```

- 2 Columns have Missing Values
- No duplicates
- Now ,I want to perform some Preprocessing Techniques on columns like:

Handling Missing values
Handling Inconsistence Data
Data Type Conversions

Checking Null Values After Preprocessing

```
df.isnull().sum()
```

brand	0
model	0
range_(km)	0
efficiency_(wh/km)	0
battery_(kwh)	0
fastcharge_speed_(kw)	0
weight_(kg)	0
acceleration_(0_100s)	0
tow_weight_(kg)	53
cargo_(l)	22
long_distance_(km)	0
price/range_(€_per_km)	0
market_segment	0
seats	0
dtype:	int64

```
df.isnull().sum()
```

brand	0
model	0
range_(km)	0
efficiency_(wh/km)	0
battery_(kwh)	0
fastcharge_speed_(kw)	0
weight_(kg)	0
acceleration_(0_100s)	0
tow_weight_(kg)	0
cargo_(l)	0
long_distance_(km)	0
price/range_(€_per_km)	0
market_segment	0
seats	0
dtype:	int64

After preprocessing of the Dataset we have filled the missing values with median because it doesn't effected by the Outliers due to this the dataset will be more consistent


```
df.describe()
```

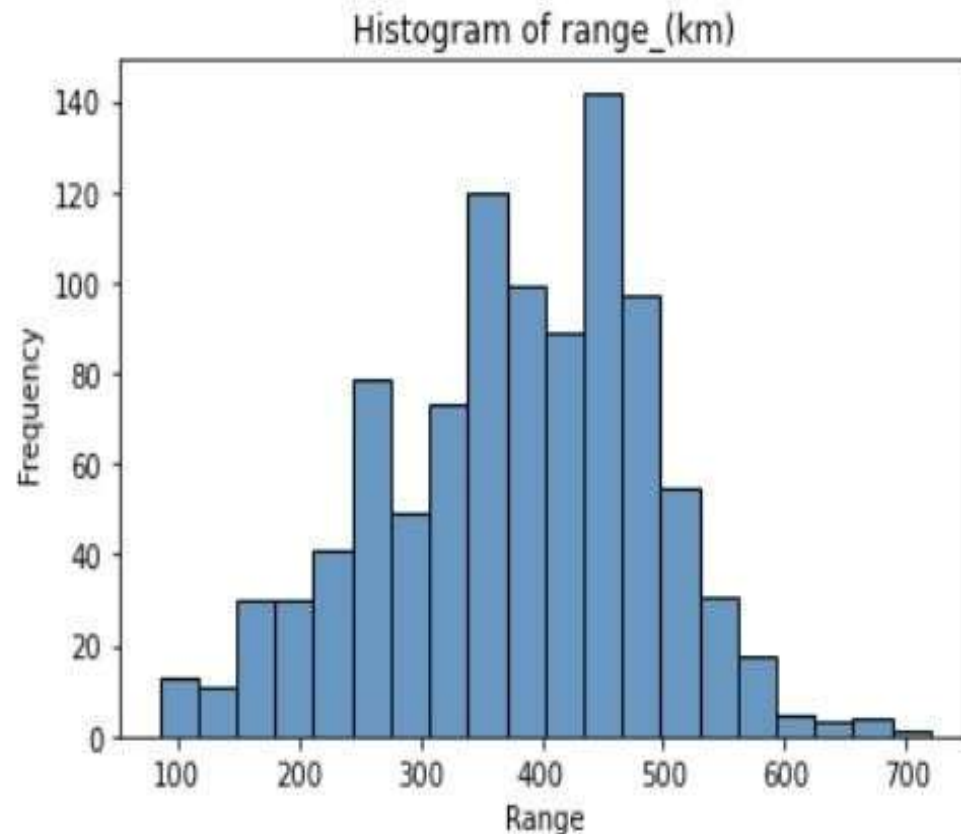
	range_(km)	efficiency_(wh/km)	battery_(kwh)	fastcharge_speed_(kw)	weight_(kg)	acceleration_(0_100s)	tow_weight_(kg)	cargo_(l)	long_distance_(km)	price
count	990.00000	990.000000	990.000000	990.000000	990.000000	990.000000	990.000000	990.000000	990.000000	
mean	377.70202	190.877778	71.445354	114.278788	2101.316162	7.183030	1023.280683	536.695248	412.308081	
std	110.67331	28.773658	21.087983	54.690443	357.069737	2.961819	717.660784	213.624731	140.681388	
min	85.00000	109.000000	14.500000	0.000000	1009.000000	2.000000	0.000000	116.000000	65.000000	
25%	305.00000	170.000000	57.550000	78.000000	1881.000000	4.900000	500.000000	430.000000	315.250000	
50%	385.00000	185.000000	74.400000	110.000000	2144.500000	6.700000	1000.000000	520.000000	416.500000	
75%	458.75000	204.000000	87.650000	138.000000	2348.000000	8.600000	1600.000000	603.000000	514.000000	
max	720.00000	295.000000	123.000000	330.000000	3085.000000	22.400000	3000.000000	2000.000000	836.000000	

After preprocessing of the Dataset,Cleaned Dataset is Saved as uber.csv.
Now Dataset is ready to analyze and find Valuable insights.

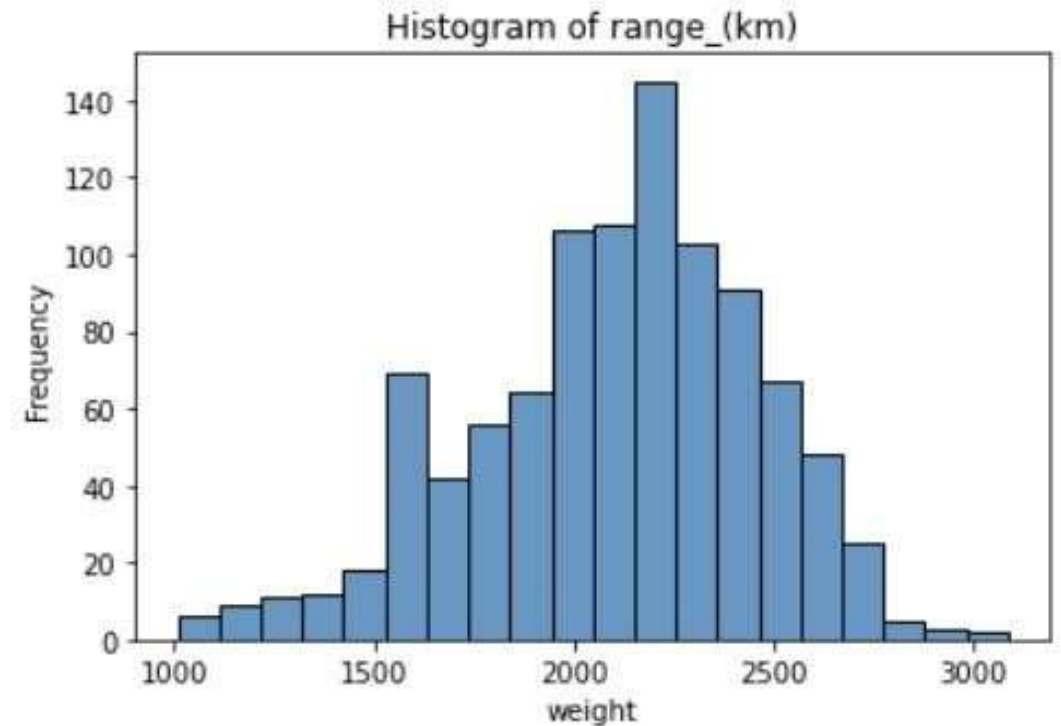
Data Analysis through Visualizations

Univariate Analysis

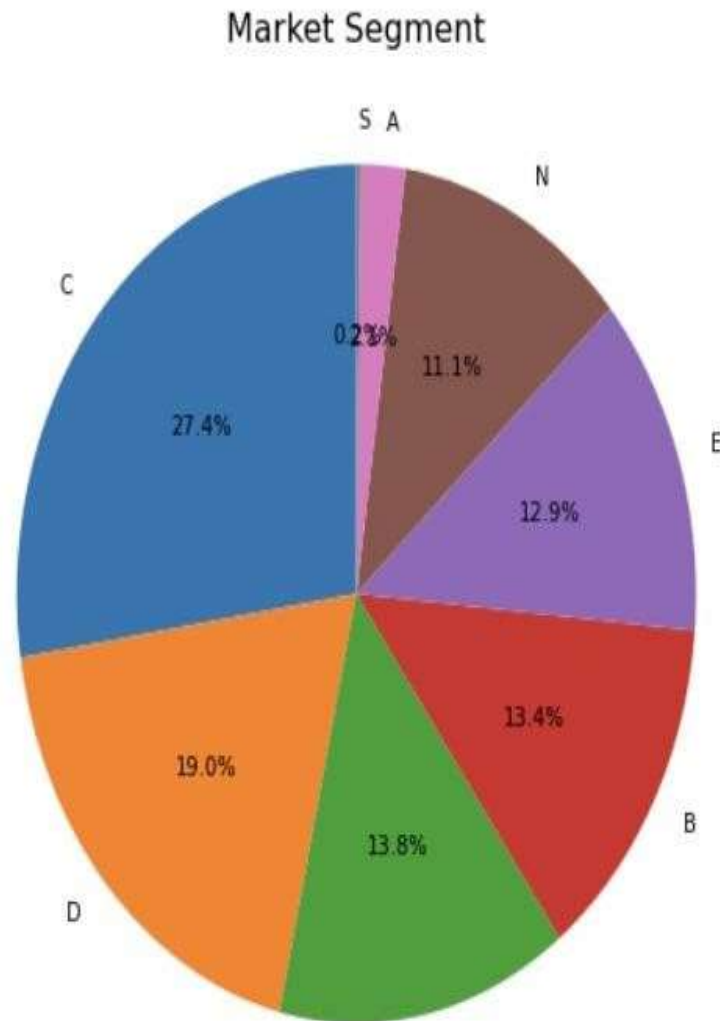
```
sns.histplot(df["range_(km)"], bins=20, kde=False)  
plt.title("Histogram of range_(km)")  
plt.xlabel("Range")  
plt.ylabel("Frequency")  
plt.show()
```



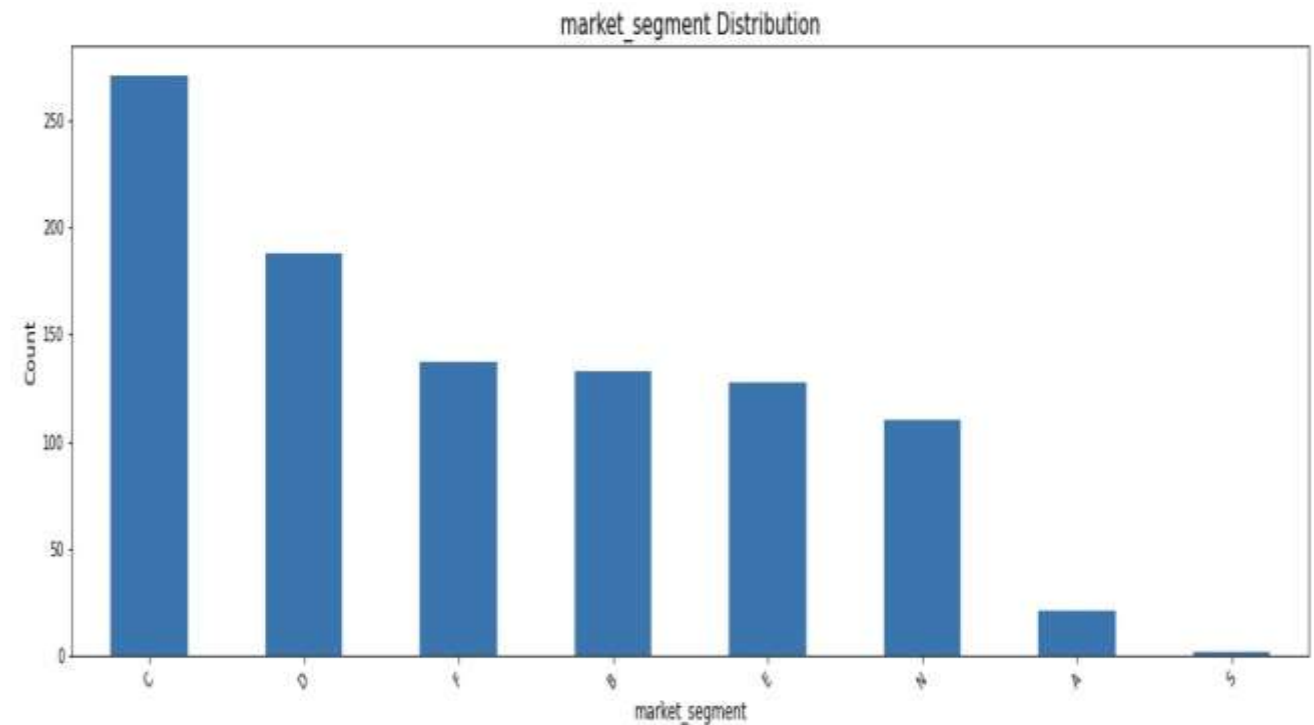
```
sns.histplot(df["weight_(kg)"], bins=20, kde=False)  
plt.title("Histogram of weight")  
plt.xlabel("weight")  
plt.ylabel("Frequency")  
plt.show()
```



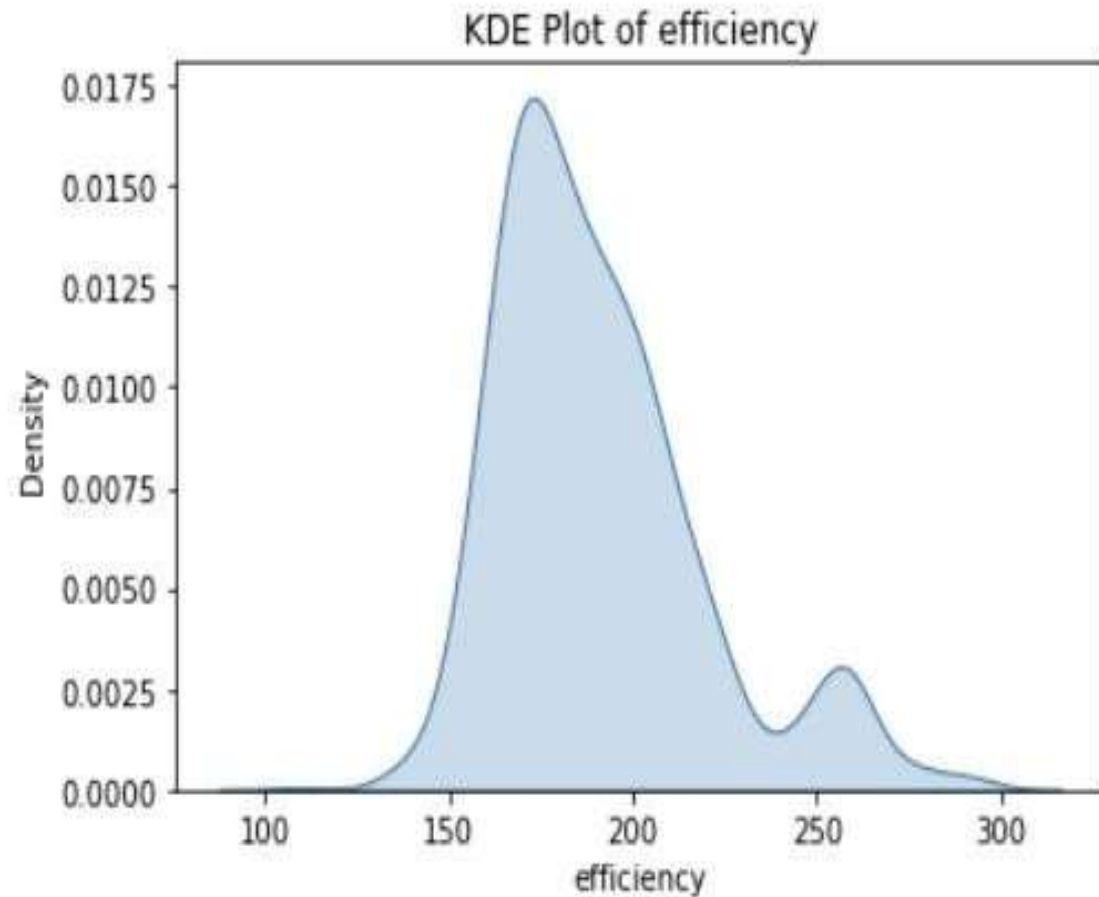
```
plt.figure(figsize=(8, 8)) |
df["market_segment"].value_counts().plot.pie(autopct="%1.1f%%", startangle=90)
plt.title("Market Segment", fontsize=16)
plt.ylabel("")
plt.show()
```



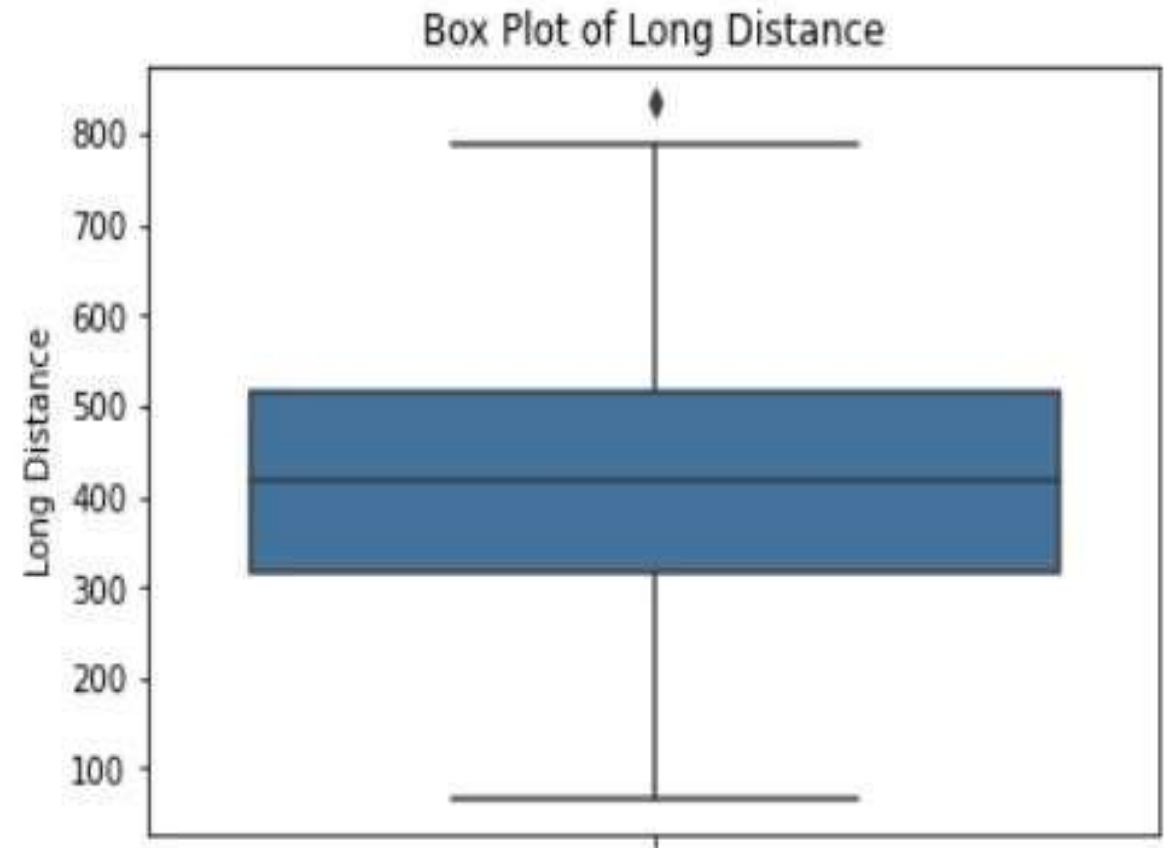
```
plt.figure(figsize=(20, 6)) # width, height in inches
df["market_segment"].value_counts().plot(kind="bar")
plt.title("market_segment Distribution", fontsize=16)
plt.xlabel("market_segment", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.xticks(rotation=45)
plt.show()
```



```
sns.kdeplot(df["efficiency_(wh/km)"], fill=True)
plt.title("KDE Plot of efficiency")
plt.xlabel("efficiency")
plt.show()
```

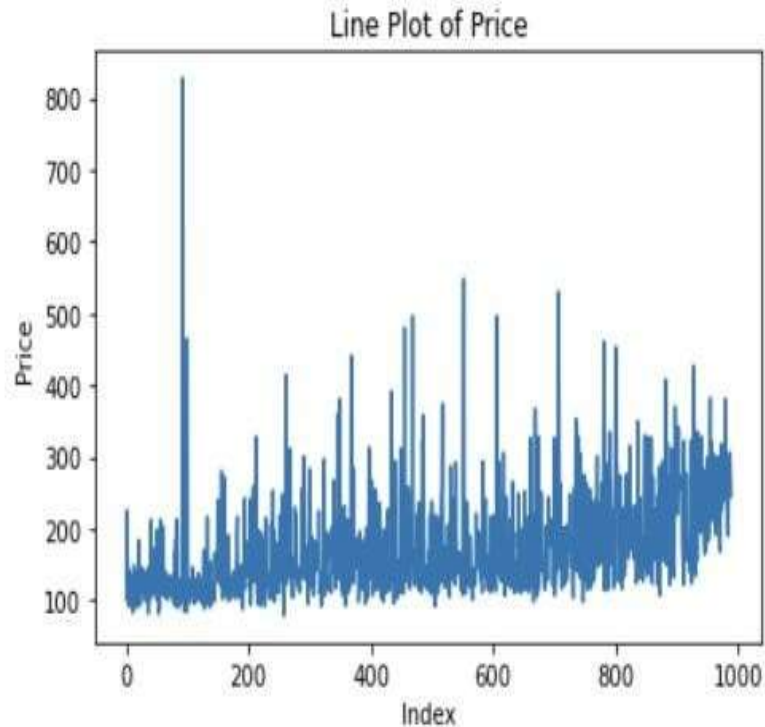


```
sns.boxplot(y=df["long_distance_(km)"])
plt.title("Box Plot of Long Distance")
plt.ylabel("Long Distance")
plt.show()
```

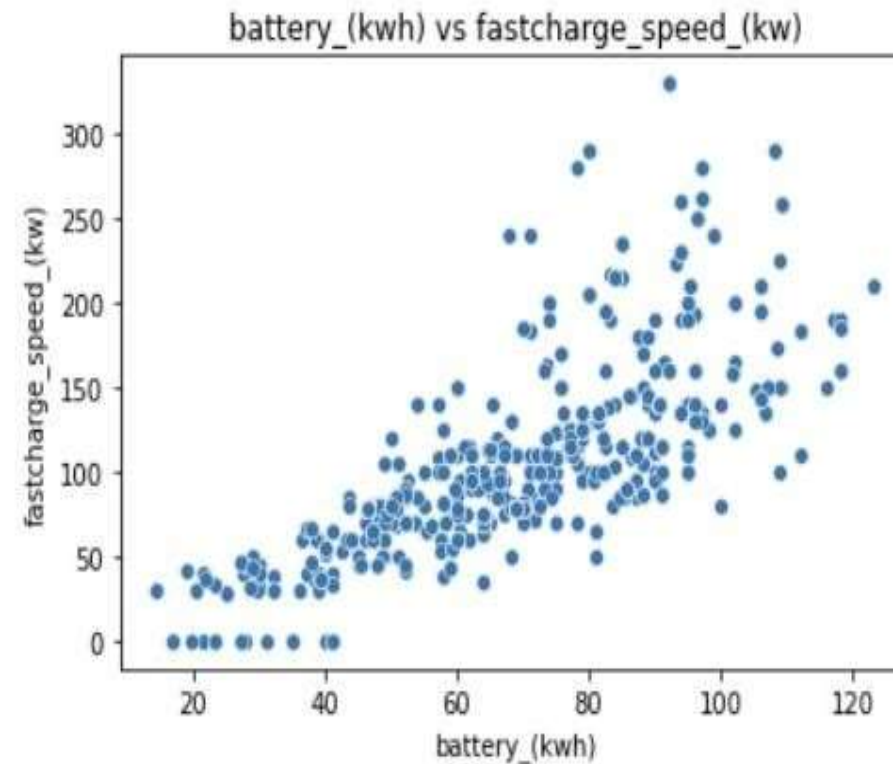


Bivariate Analysis-Categorical and Numerical

```
In [23]: plt.plot(df["price/range_(€_per_km)"])
plt.title("Line Plot of Price")
plt.xlabel("Price")
plt.ylabel("Range")
plt.show()
```

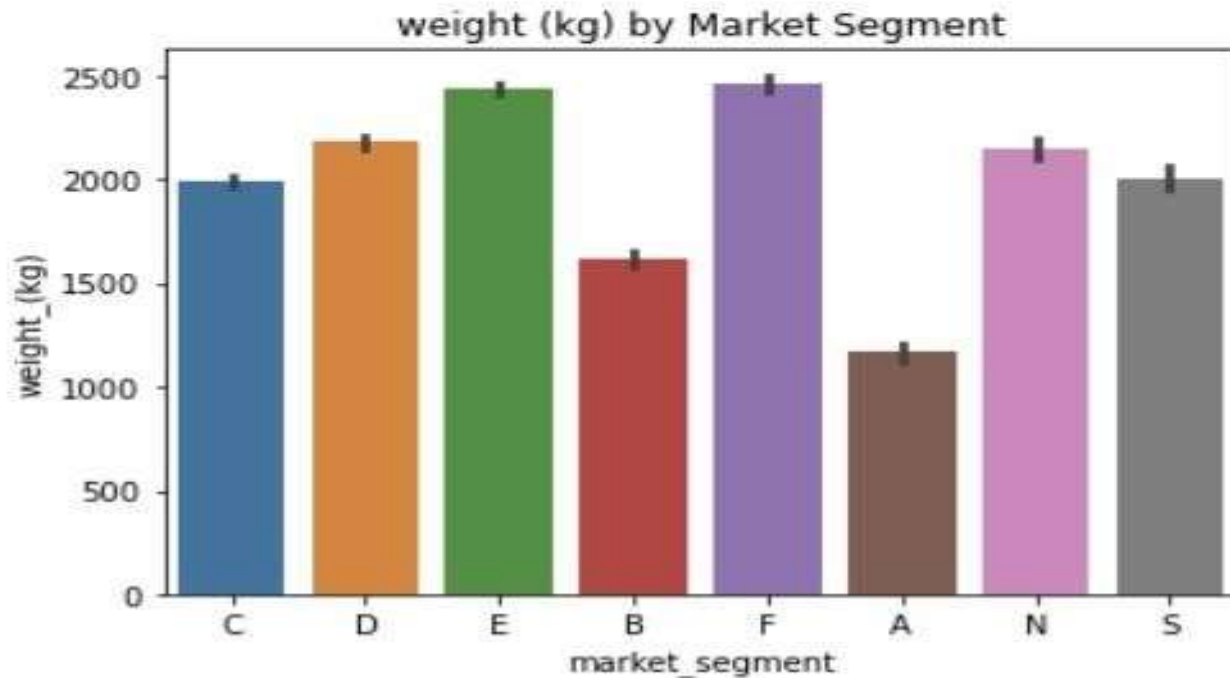


```
sns.scatterplot(x='battery_(kwh)', y='fastcharge_speed_(kw)', data=df)
plt.title("battery_(kwh) vs fastcharge_speed_(kw)")
plt.xlabel("battery_(kwh)")
plt.ylabel("fastcharge_speed_(kw)")
plt.show()
```

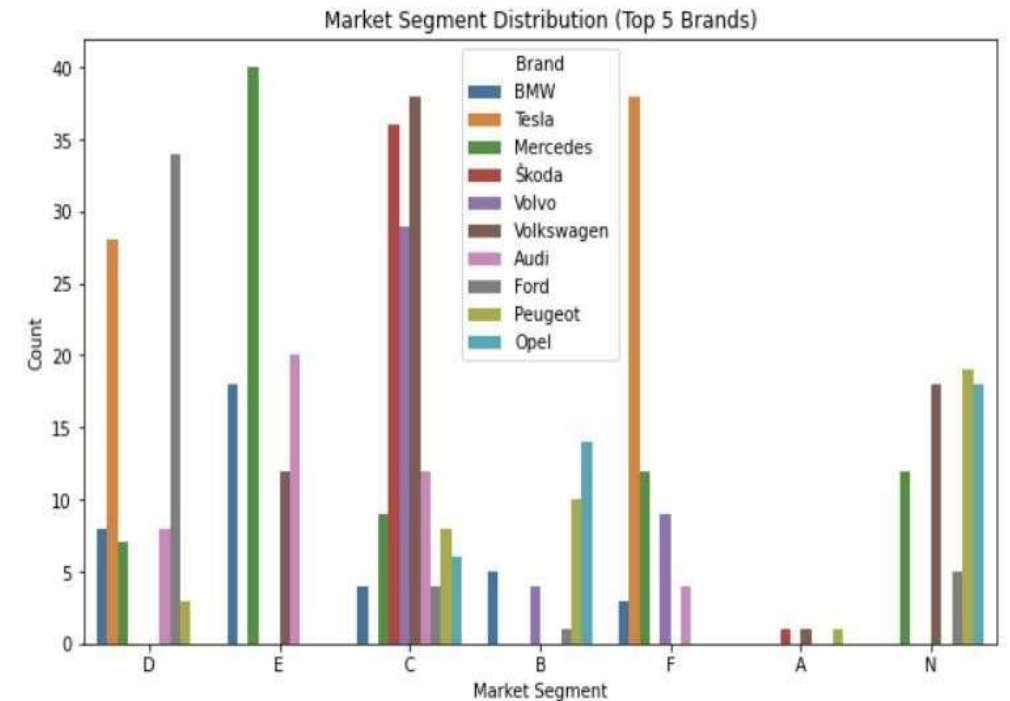


Multivariate Analysis

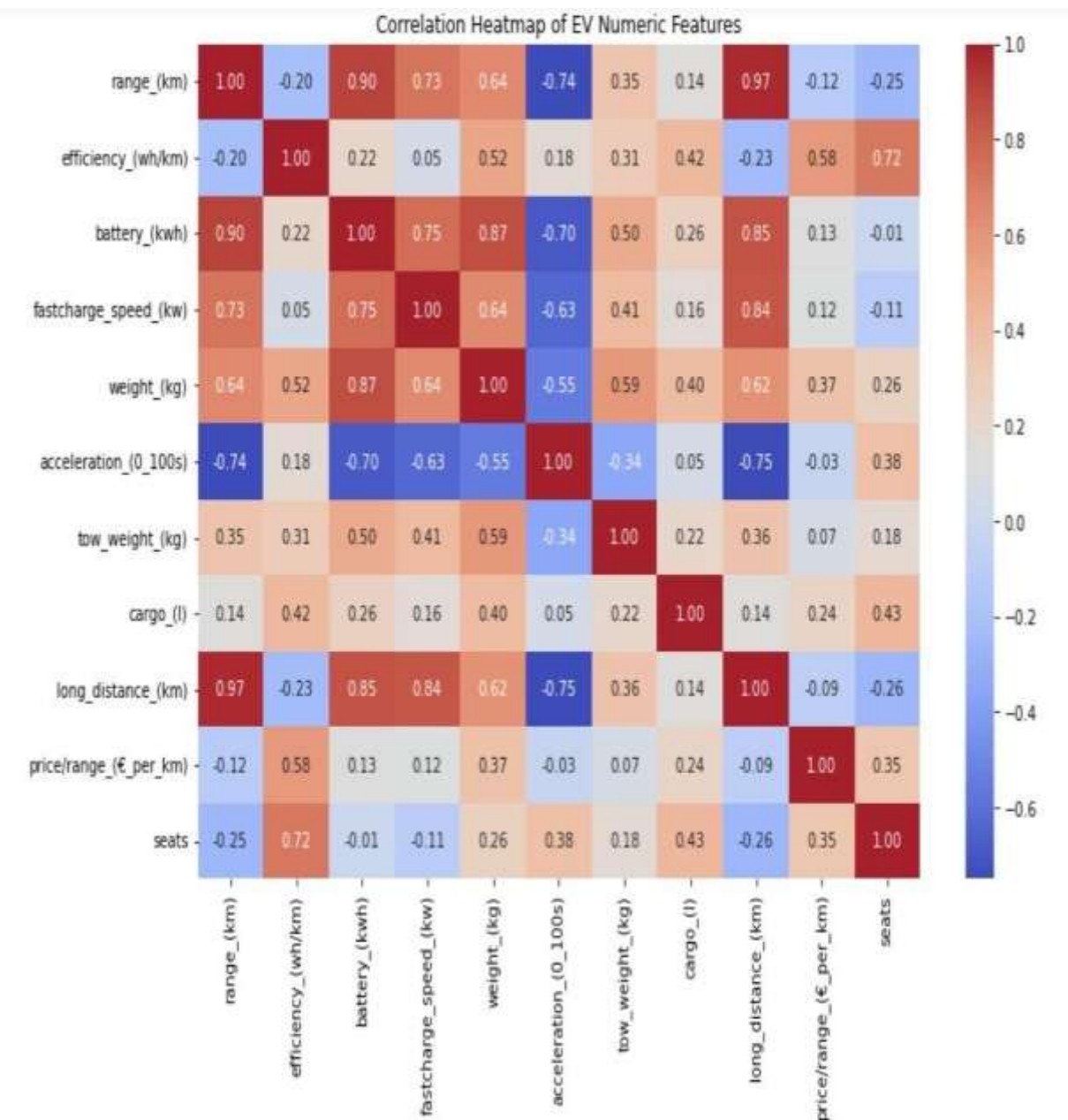
```
sns.barplot(  
    x="market_segment",  
    y="weight_(kg)",  
    data=df  
)  
  
plt.title("weight (kg) by Market Segment")  
plt.show()
```



```
top_brands = df["brand"].value_counts().head(10).index  
filtered_df = df[df["brand"].isin(top_brands)]  
plt.figure(figsize=(10, 6))  
sns.countplot(x="market_segment", hue="brand", data=filtered_df)  
  
plt.title("Market Segment Distribution (Top 10 Brands)")  
plt.xlabel("Market Segment")  
plt.ylabel("Count")  
plt.legend(title="Brand")  
plt.show()
```



HEATMAP



Summary

- Fast charging speeds show high variability across EV models
- Battery capacity and long-distance range contain visible outliers
- Acceleration distribution is right-skewed, indicating performance differences
- Efficiency follows a near-normal distribution with slight multi-modality
- Most EVs fall within a mid-range weight and driving range segment

THANK
YOU

